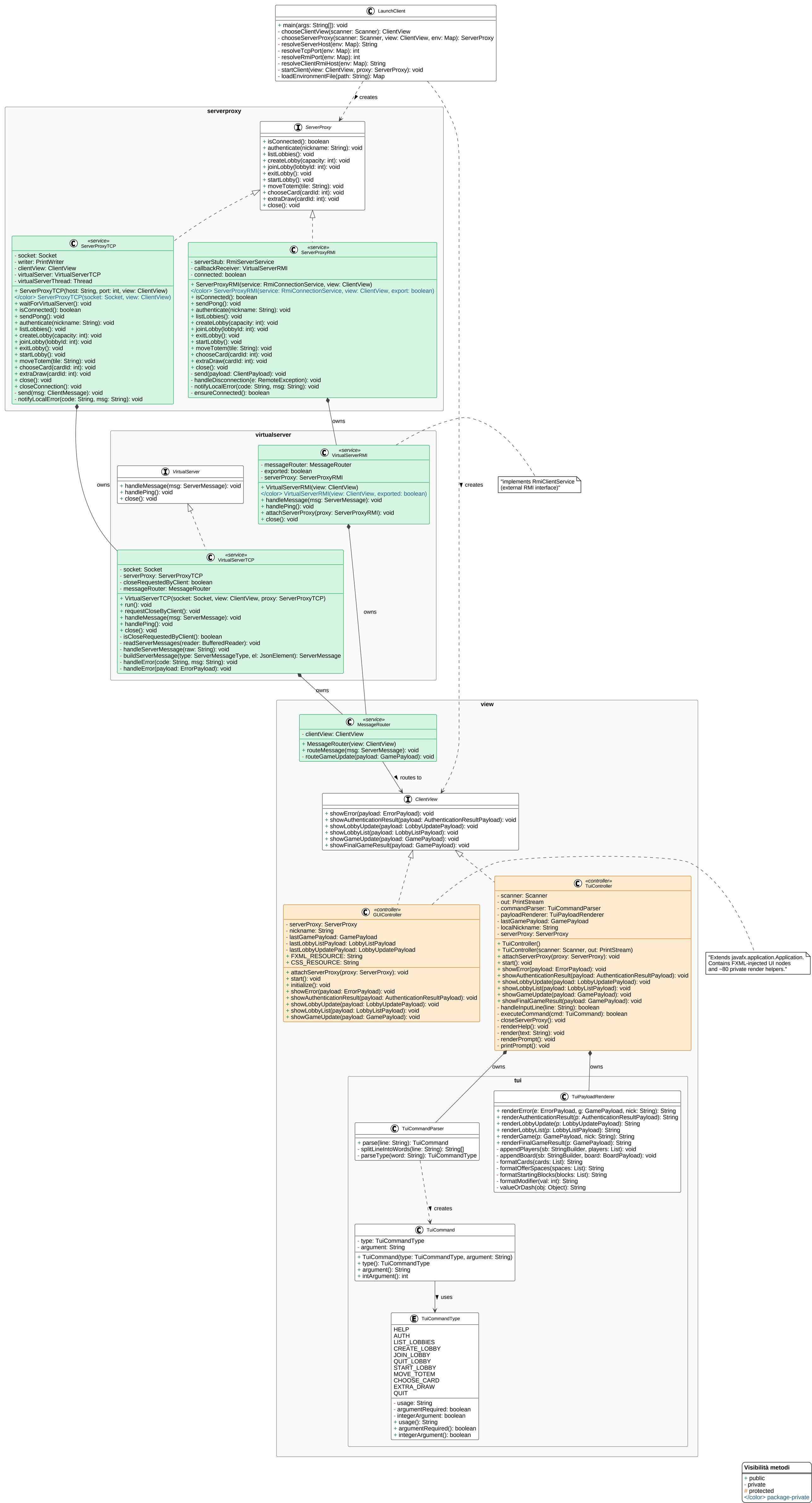
LaunchClient
+ main(args: String[]): void
- chooseClientView(scanner: Scanner): ClientView
- chooseServerProxy(scanner: Scanner, view: ClientView, env: Map): ServerProxy
- resolveServerHost(env: Map): String
- resolveTcpPort(env: Map): int
- resolveRmiPort(env: Map): int
- resolveClientRmiHost(env: Map): String
- startClient(view: ClientView, proxy: ServerProxy): void
- loadEnvironmentFile(path: String): Map
creates
serverproxy
ServerProxy
+ isConnected(): boolean
+ authenticate(nickname: String): void
+ listLobbies(): void
+ createLobby(capacity: int): void
+ joinLobby(lobbyId: int): void
+ exitLobby(): void
+ startLobby(): void
+ moveTotem(tile: String): void
+ chooseCard(cardId: int): void
+ extraDraw(cardId: int): void
+ close(): void
«service»
ServerProxyTCP
- socket: Socket
- writer: PrintWriter
- clientView: ClientView
- virtualServer: VirtualServerTCP
- virtualServerThread: Thread
+ ServerProxyTCP(host: String, port: int, view: ClientView)
</color> ServerProxyTCP(socket: Socket, view: ClientView)
+ waitForVirtualServer(): void
+ isConnected(): boolean
+ sendPong(): void
+ authenticate(nickname: String): void
+ listLobbies(): void
+ createLobby(capacity: int): void
+ joinLobby(lobbyId: int): void
+ exitLobby(): void
+ startLobby(): void
+ moveTotem(tile: String): void
+ chooseCard(cardId: int): void
+ extraDraw(cardId: int): void
+ close(): void
+ closeConnection(): void
- send(msg: ClientMessage): void
- notifyLocalError(code: String, msg: String): void
«service»
ServerProxyRMI
- serverStub: RmiServerService
- callbackReceiver: VirtualServerRMI
- connected: boolean
+ ServerProxyRMI(service: RmiConnectionService, view: ClientView)
</color> ServerProxyRMI(service: RmiConnectionService, view: ClientView, export: boolean)
+ isConnected(): boolean
+ sendPong(): void
+ authenticate(nickname: String): void
+ listLobbies(): void
+ createLobby(capacity: int): void
+ joinLobby(lobbyId: int): void
+ exitLobby(): void
+ startLobby(): void
+ moveTotem(tile: String): void
+ chooseCard(cardId: int): void
+ extraDraw(cardId: int): void
+ close(): void
- send(payload: ClientPayload): void
- handleDisconnection(e: RemoteException): void
- notifyLocalError(code: String, msg: String): void
- ensureConnected(): boolean
owns
virtualserver
«service»
VirtualServerRMI
- messageRouter: MessageRouter
- exported: boolean
- serverProxy: ServerProxyRMI
+ VirtualServerRMI(view: ClientView)
</color> VirtualServerRMI(view: ClientView, exported: boolean)
+ handleMessage(msg: ServerMessage): void
+ handlePing(): void
+ attachServerProxy(proxy: ServerProxyRMI): void
+ close(): void
creates
"implements RmiClientService
(external RMI interface)"
VirtualServer
+ handleMessage(msg: ServerMessage): void
+ handlePing(): void
+ close(): void
owns
«service»
VirtualServerTCP
- socket: Socket
- serverProxy: ServerProxyTCP
- closeRequestedByClient: boolean
- messageRouter: MessageRouter
+ VirtualServerTCP(socket: Socket, view: ClientView, proxy: ServerProxyTCP)
+ run(): void
+ requestCloseByClient(): void
+ handleMessage(msg: ServerMessage): void
+ handlePing(): void
+ close(): void
- isCloseRequestedByClient(): boolean
- readServerMessages(reader: BufferedReader): void
- handleServerMessage(raw: String): void
- buildServerMessage(type: ServerMessageType, el: JsonElement): ServerMessage
- handleError(code: String, msg: String): void
- handleError(payload: ErrorPayload): void
owns
owns
view
«service»
MessageRouter
- clientView: ClientView
+ MessageRouter(view: ClientView)
+ routeMessage(msg: ServerMessage): void
- routeGameUpdate(payload: GamePayload): void
routes to
ClientView
+ showError(payload: ErrorPayload): void
+ showAuthenticationResult(payload: AuthenticationResultPayload): void
+ showLobbyUpdate(payload: LobbyUpdatePayload): void
+ showLobbyList(payload: LobbyListPayload): void
+ showGameUpdate(payload: GamePayload): void
+ showFinalGameResult(payload: GamePayload): void
«controller»
GUIController
- serverProxy: ServerProxy
- nickname: String
- lastGamePayload: GamePayload
- lastLobbyListPayload: LobbyListPayload
- lastLobbyUpdatePayload: LobbyUpdatePayload
+ FXML_RESOURCE: String
+ CSS_RESOURCE: String
+ attachServerProxy(proxy: ServerProxy): void
+ start(): void
+ initialize(): void
+ showError(payload: ErrorPayload): void
+ showAuthenticationResult(payload: AuthenticationResultPayload): void
+ showLobbyUpdate(payload: LobbyUpdatePayload): void
+ showLobbyList(payload: LobbyListPayload): void
+ showGameUpdate(payload: GamePayload): void
«controller»
TuiController
- scanner: Scanner
- out: PrintStream
- commandParser: TuiCommandParser
- payloadRenderer: TuiPayloadRenderer
- lastGamePayload: GamePayload
- localNickname: String
- serverProxy: ServerProxy
+ TuiController()
+ TuiController(scanner: Scanner, out: PrintStream)
+ attachServerProxy(proxy: ServerProxy): void
+ start(): void
+ showError(payload: ErrorPayload): void
+ showAuthenticationResult(payload: AuthenticationResultPayload): void
+ showLobbyUpdate(payload: LobbyUpdatePayload): void
+ showLobbyList(payload: LobbyListPayload): void
+ showGameUpdate(payload: GamePayload): void
+ showFinalGameResult(payload: GamePayload): void
- handleInputLine(line: String): boolean
- executeCommand(cmd: TuiCommand): boolean
- closeServerProxy(): void
- renderHelp(): void
- render(text: String): void
- renderPrompt(): void
- printPrompt(): void
"Extends javafx.application.Application.
Contains FXML-injected UI nodes
and ~80 private render helpers."
owns
owns
tui
TuiCommandParser
+ parse(line: String): TuiCommand
- splitLineIntoWords(line: String): String[]
- parseType(word: String): TuiCommandType
TuiPayloadRenderer
+ renderError(e: ErrorPayload, g: GamePayload, nick: String): String
+ renderAuthenticationResult(p: AuthenticationResultPayload): String
+ renderLobbyUpdate(p: LobbyUpdatePayload): String
+ renderLobbyList(p: LobbyListPayload): String
+ renderGame(p: GamePayload, nick: String): String
+ renderFinalGameResult(p: GamePayload): String
- appendPlayers(sb: StringBuilder, players: List): void
- appendBoard(sb: StringBuilder, board: BoardPayload): void
- formatCards(cards: List): String
- formatOfferSpaces(spaces: List): String
- formatStartingBlocks(blocks: List): String
- formatModifier(val: int): String
- valueOrDash(obj: Object): String
creates
TuiCommand
- type: TuiCommandType
- argument: String
+ TuiCommand(type: TuiCommandType, argument: String)
+ type(): TuiCommandType
+ argument(): String
+ intArgument(): int
uses
TuiCommandType
HELP
AUTH
LIST_LOBBIES
CREATE_LOBBY
JOIN_LOBBY
QUIT_LOBBY
START_LOBBY
MOVE_TOTEM
CHOOSE_CARD
EXTRA_DRAW
QUIT
- usage: String
- argumentRequired: boolean
- integerArgument: boolean
+ usage(): String
+ argumentRequired(): boolean
+ integerArgument(): boolean
Visibilità metodi
+ public
- private
protected
</color> package-private